Universidad Nacional Experimental De Guayana
Vicerrectorado Académico
Coordinación General De Pregrado
P.F.G: Ingeniera En Informática
Periodo académico: 2026 - I
Unidad Curricular: Bases de datos II
Sección 1 y 2

PROYECTO III - Logistics Optimizer (Smart Routing)

Facilitador:

Ing. Clinia Cordero

Bachiller:

Angel Gonzalez - C.I: 30.366.096

Glihanny Sotillo - C.I: 27.766.805

Cesar Reyes - C.I: 29.906.628

Ciudad Guayana, Marzo del 2026

Introducción

Este proyecto enfrenta el problema de optimizar rutas en un contexto logístico, donde se requiere hallar el recorrido más eficiente entre almacenes y puntos de entrega a partir de varios factores, como distancia, tiempo estimado y condiciones de tráfico.

En sistemas logísticos reales, decidir solamente en función de la distancia puede no ser suficiente, pues elementos como la congestión del tráfico o las limitaciones de carga tienen un impacto directo en la eficacia del transporte. Por eso, es necesario emplear modelos que hagan posible la optimización de rutas a partir de diferentes criterios.

Se empleó un enfoque que se basa en grafos para modelar este problema, en el cual los puntos de entrega, las intersecciones y los almacenes son representados como nodos, y las conexiones entre estos como relaciones ponderadas. Esta perspectiva posibilita el uso de algoritmos de caminos mínimos para calcular las rutas más adecuadas.

El propósito del proyecto es el de comparar rutas óptimas utilizando diferentes criterios de optimización, como la distancia y el tiempo estimado. Para ello, se hace uso de algoritmos de teoría de grafos implementados en una proyección en memoria a través de la librería Graph Data Science.

Modelado de grafos

Tipos de nodos

- Almacen
- Interseccion
- PuntoEntrega

Propiedades relevantes

Nodo	Propiedades
Almacen	id, nombre, capacidad, lat, lon
Interseccion	id, nombre, lat, lon
PuntoEntrega	id, lat, lon

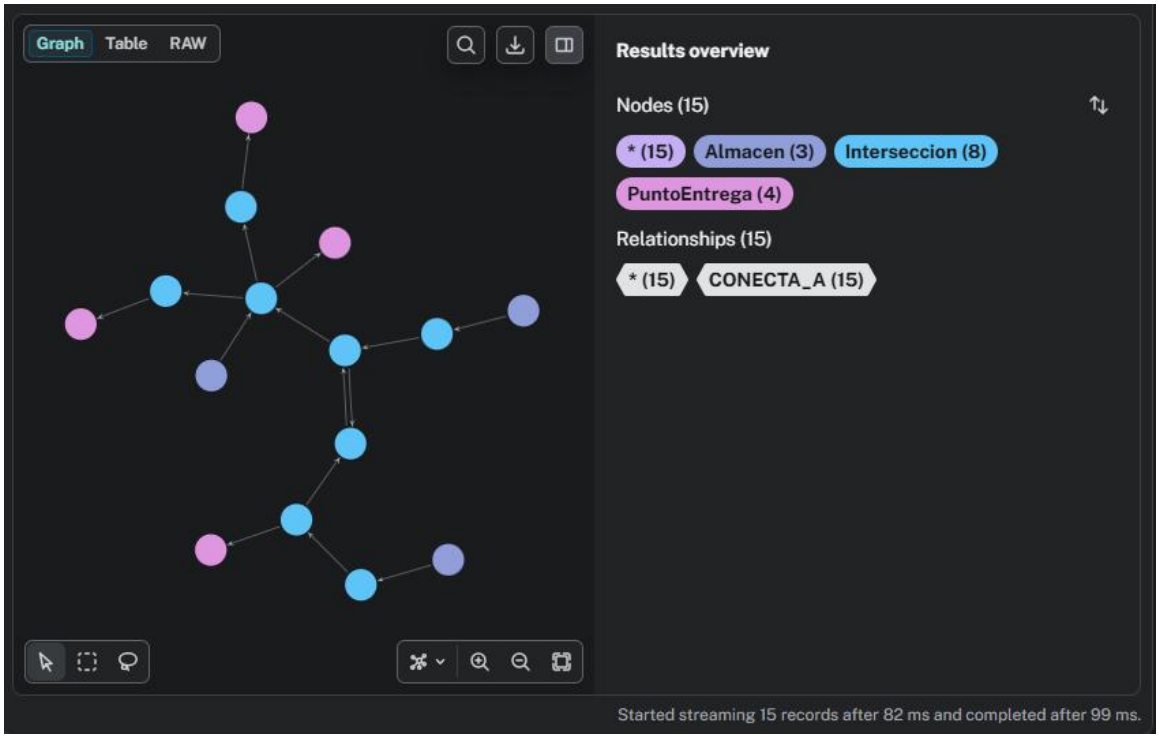
Relaciones

conecta_a (representa calles o rutas)

Propiedades:

distancia
tiempo_estimado
estado_trafico
peso_maximo

Visualización del grafo entero



Carga de datos

Primera celda, limpieza de Base de datos

```
/*
==========================================================
PROYECTO III - Logistics Optimizer (Smart Routing)

Integrantes:
Angel Gonzalez (S1) - C.I: 30.366.096
Glihanny Sotillo (S2) - C.I: 27.766.805
Cesar Reyes (S1) - C.I: 29.906.628
==========================================================
*/

//Primera parte a ejecutar

// ========================================================
// LIMPIEZA DE DATOS
// ========================================================

MATCH (n)
```

```
DETACH DELETE n;
```

Segunda celda, creacion de restricciones de unicidad.

```
// ==========================================================
// RESTRICCIONES DE UNICIDAD
// ==========================================================

CREATE CONSTRAINT almacen_id IF NOT EXISTS
FOR (a:Almacen)
REQUIRE a.id IS UNIQUE;

CREATE CONSTRAINT punto_id IF NOT EXISTS
FOR (p:PuntoEntrega)
REQUIRE p.id IS UNIQUE;

CREATE CONSTRAINT interseccion_id IF NOT EXISTS
FOR (i:Interseccion)
REQUIRE i.id IS UNIQUE;
```

Tercera celda, creacion de nodos a conectar.

```
// ==========================================================
// CREACIÓN DE NODOS
// ==========================================================

CREATE
// ALMACENES
(a1:Almacen {id:"A1", nombre:"Almacen Central", capacidad:100, lat:8.289, lon:-
62.719}),
(a2:Almacen {id:"A2", nombre:"Almacen Norte", capacidad:80, lat:8.300, lon:-62.700}),
(a3:Almacen {id:"A3", nombre:"Almacen Sur", capacidad:60, lat:8.270, lon:-62.730}),

// PUNTOS DE ENTREGA
(p1:PuntoEntrega {id:"P1", nombre:"Cliente 1", lat:8.310, lon:-62.710}),
(p2:PuntoEntrega {id:"P2", nombre:"Cliente 2", lat:8.320, lon:-62.690}),
(p3:PuntoEntrega {id:"P3", nombre:"Cliente 3", lat:8.305, lon:-62.725}),
(p4:PuntoEntrega {id:"P4", nombre:"Cliente 4", lat:8.295, lon:-62.740}),

// INTERSECCIONES
(i1:Interseccion {id:"I1", lat:8.291, lon:-62.715}),
```

(i2:Interseccion {id:"I2", lat:8.295, lon:-62.712}),
(i3:Interseccion {id:"I3", lat:8.305, lon:-62.705}),
(i4:Interseccion {id:"I4", lat:8.315, lon:-62.698}),
(i5:Interseccion {id:"I5", lat:8.280, lon:-62.725}),
(i6:Interseccion {id:"I6", lat:8.275, lon:-62.735}),
(i7:Interseccion {id:"I7", lat:8.298, lon:-62.720}),
(i8:Interseccion {id:"I8", lat:8.310, lon:-62.720});
Cuarta, conexion de nodos

Cuarta celda, conexiones.

```
// ====================================
// CREACIÓN DE RELACIONES CONECTA_A
// ====================================

MATCH
(a1:Almacen {id:"A1"}),
(a2:Almacen {id:"A2"}),
(a3:Almacen {id:"A3"}),
(p1:PuntoEntrega {id:"P1"}),
(p2:PuntoEntrega {id:"P2"}),
(p3:PuntoEntrega {id:"P3"}),
(p4:PuntoEntrega {id:"P4"}),
(i1:Interseccion {id:"I1"}),
(i2:Interseccion {id:"I2"}),
(i3:Interseccion {id:"I3"}),
(i4:Interseccion {id:"I4"}),
(i5:Interseccion {id:"I5"}),
(i6:Interseccion {id:"I6"}),
(i7:Interseccion {id:"I7"}),
(i8:Interseccion {id:"I8"})

CREATE

// ================================
// CONEXIÓN ALMACENES A LA RED
// ================================
(a1)-[:CONECTA_A {distancia:5, tiempo_estimado:10, estado_trafico:0.3,
peso_maximo:20}]->(i1),
```

```
(a2)-[:CONECTA_A {distancia:6, tiempo_estimado:12, estado_trafico:0.6,
peso_maximo:10}]->(i3),
(a3)-[:CONECTA_A {distancia:4, tiempo_estimado:7, estado_trafico:0.2,
peso_maximo:25}]->(i5),

// ================================
// RED CENTRAL DE INTERSECCIONES
// ================================

// Eje horizontal principal
(i1)-[:CONECTA_A {distancia:2, tiempo_estimado:4, estado_trafico:0.3,
peso_maximo:25}]->(i2),
(i2)-[:CONECTA_A {distancia:2, tiempo_estimado:4, estado_trafico:0.4,
peso_maximo:25}]->(i3),
(i3)-[:CONECTA_A {distancia:2, tiempo_estimado:5, estado_trafico:0.5,
peso_maximo:25}]->(i4),

// Eje vertical
(i2)-[:CONECTA_A {distancia:3, tiempo_estimado:6, estado_trafico:0.3,
peso_maximo:20}]->(i7),
(i3)-[:CONECTA_A {distancia:3, tiempo_estimado:6, estado_trafico:0.6,
peso_maximo:20}]->(i8),

// Zona sur
(i5)-[:CONECTA_A {distancia:2, tiempo_estimado:4, estado_trafico:0.2,
peso_maximo:20}]->(i6),
(i6)-[:CONECTA_A {distancia:3, tiempo_estimado:5, estado_trafico:0.3,
peso_maximo:20}]->(i7),

// Conexión norte-sur
(i7)-[:CONECTA_A {distancia:2, tiempo_estimado:4, estado_trafico:0.5,
peso_maximo:20}]->(i2),

// ================================
// CONEXIÓN A CLIENTES
// ================================

(i4)-[:CONECTA_A {distancia:3, tiempo_estimado:6, estado_trafico:0.4,
peso_maximo:20}]->(p2),
(i3)-[:CONECTA_A {distancia:4, tiempo_estimado:8, estado_trafico:0.2,
peso_maximo:25}]->(p1),
(i6)-[:CONECTA_A {distancia:2, tiempo_estimado:5, estado_trafico:0.1,
peso_maximo:15}]->(p3),
```

```
(i8)-[:CONECTA_A {distancia:2, tiempo_estimado:4, estado_trafico:0.3,
peso_maximo:20}]->(p4);
```

Quinta celda, agregado extra de visualización del grafo.

```
// ==========================================
// MOSTRAR TODAS LAS RELACIONES
// ==========================================

MATCH (a)-[r:CONECTA_A]->(b)
RETURN a, r, b,
    r.distancia AS distancia,
    r.tiempo_estimado AS tiempo,
    r.estado_trafico AS trafico,
    r.peso_maximo AS peso;
```

Proyección en memoria

Crear una representación temporal del grafo en la RAM para llevar a cabo algoritmos de análisis sin impactar el almacenamiento persistente es lo que se conoce como proyección en memoria. En vez de trabajar directamente con los datos de las transacciones, se crea una estructura optimizada para el cálculo intensivo; esto aumenta notablemente la eficiencia en las consultas analíticas.

Uso de Neo4j Graph Data Science

La librería Neo4j Graph Data Science posibilita la implementación de algoritmos sofisticados en grafos, tales como detección de comunidades, centralidad y caminos mínimos. Su diseño está orientado a análisis computacionales y opera únicamente sobre grafos que están proyectados en memoria, dividiendo el procesamiento analítico del motor de transacciones de Neo4j.

Creación de proyección con gds.graph.project

La función gds.graph.project crea un grafo en la memoria, indicando cuáles serán las relaciones, los nodos y las propiedades que se incluirán. Este proceso copia los datos guardados en la base, los traslada a una estructura optimizada en RAM y le asigna un nombre identificador al grafo proyectado para que después los algoritmos puedan funcionar sobre él.

Ejecución de algoritmos sobre grafo en memoria

Luego de que la proyección ha sido elaborada, se llevan a cabo de manera directa en la estructura en memoria los algoritmos como A* o Dijkstra. Esto posibilita hacer cálculos eficaces de rutas óptimas teniendo en cuenta atributos como la distancia o el tiempo estimado. El resultado se devuelve en forma de flujo de datos (stream), manteniendo sin cambios los datos originales que están guardados en la base.

Diccionario de consultas

Consulta 1 - Ruta minima por distancia (Dijkstra)

Objetivo: calcular ruta minima por distancia mediante el algoritmo Dijkstra

Codigo:

```
// ==========================================
// DIJKSTRA (PESO = distancia)
// ==========================================

MATCH (source:Almacen {id:'A1'}),
     (target:PuntoEntrega {id:'P1'})

CALL gds.shortestPath.dijkstra.stream('logisticsGraph', {
   sourceNode: id(source),
   targetNode: id(target),
   relationshipWeightProperty: 'distancia'
})
YIELD totalCost, nodeIds

WITH totalCost,
    [nodeId IN nodeIds | gds.util.asNode(nodeId)] AS nodes

UNWIND range(0, size(nodes)-2) AS i
WITH totalCost, nodes[i] AS n1, nodes[i+1] AS n2

MATCH (n1)-[r:CONECTA_A]->(n2)

RETURN
'Dijkstra' AS algoritmo,
totalCost AS costo_total,
n1, r, n2;
```

Explicación: empleara el algoritmo de camino mínimo ponderado por la propiedad distancia, ejecutado sobre una proyección en memoria.

Resultado:

Graph Table RAW

	algoritmo	costo_total	n1	r	n2
1	"Dijkstra"	13.0	100	CONECTA_A	I1
2	"Dijkstra"	13.0	I1	CONECTA_A	I2
3	"Dijkstra"	13.0	I2	CONECTA_A	I3
4	"Dijkstra"	13.0	I3	CONECTA_A	P1

Consulta 2 - Ruta mínima por tiempo (A*)

Objetivo: calcular ruta minima por tiempo mediante el algoritmo A*

Codigo:

```
// ==========================================
// A* (PESO = tiempo_estimado)
// ==========================================

MATCH (source:Almacen {id:'A1'}),
      (target:PuntoEntrega {id:'P1'})

CALL gds.shortestPath.astar.stream('logisticsGraph', {
    sourceNode: id(source),
    targetNode: id(target),
    relationshipWeightProperty: 'tiempo_estimado',
    latitudeProperty: 'lat',
    longitudeProperty: 'lon'
})
YIELD totalCost, nodeIds
```

```
WITH totalCost,
    [nodeId IN nodeIds | gds.util.asNode(nodeId)] AS nodes

UNWIND range(0, size(nodes)-2) AS i
WITH totalCost, nodes[i] AS n1, nodes[i+1] AS n2

MATCH (n1)-[r:CONECTA_A]->(n2)

RETURN
'A*' AS algoritmo,
totalCost AS costo_total,
n1, r, n2;
```

Explicacion: Al aplicar el algoritmo A* empleara las coordenadas (lat, lon) para estimar la distancia restante al destino.

Resultado:

	algoritmo	costo_total	n1	r	n2
1	"A*"	26.0	100	CONECTA_A	I1
2	"A*"	26.0	I1	CONECTA_A	I2
3	"A*"	26.0	I2	CONECTA_A	I3
4	"A*"	26.0	I3	CONECTA_A	P1

Consulta 3 - Costo dinámico basado en tráfico

Objetivo: implementar un codigo que se base en el costo dinamico basado en trafico, mediante una funcion que lo calcule

Codigo:

```
// ==========================================
// COSTO DINÁMICO BASADO EN TRÁFICO
// ==========================================
```

```
MATCH path = (a:Almacen {id:'A1'})-[:CONECTA_A*]->(p:PuntoEntrega {id:'P1'})
WITH path,
    REDUCE(total = 0.0,
      r IN relationships(path) |
      total + (r.distancia * r.estado_trafico)
    ) AS costo_dinamico
RETURN
'Costo Dinámico' AS algoritmo,
costo_dinamico
ORDER BY costo_dinamico ASC
LIMIT 1;
```

Explicación: empleara WITH y REDUCE para calcular la suma acumulativa de distancia * estado_trafico a lo largo de un camino.

Resultado:

	algoritmo	costo_dinamico
1	"Costo Dinámico"	3.7

Consulta 4 - Restricción de peso

Objetivo: generar la mejor ruta en base al peso del camión deshabilitando rutas que no funcionen para ello.

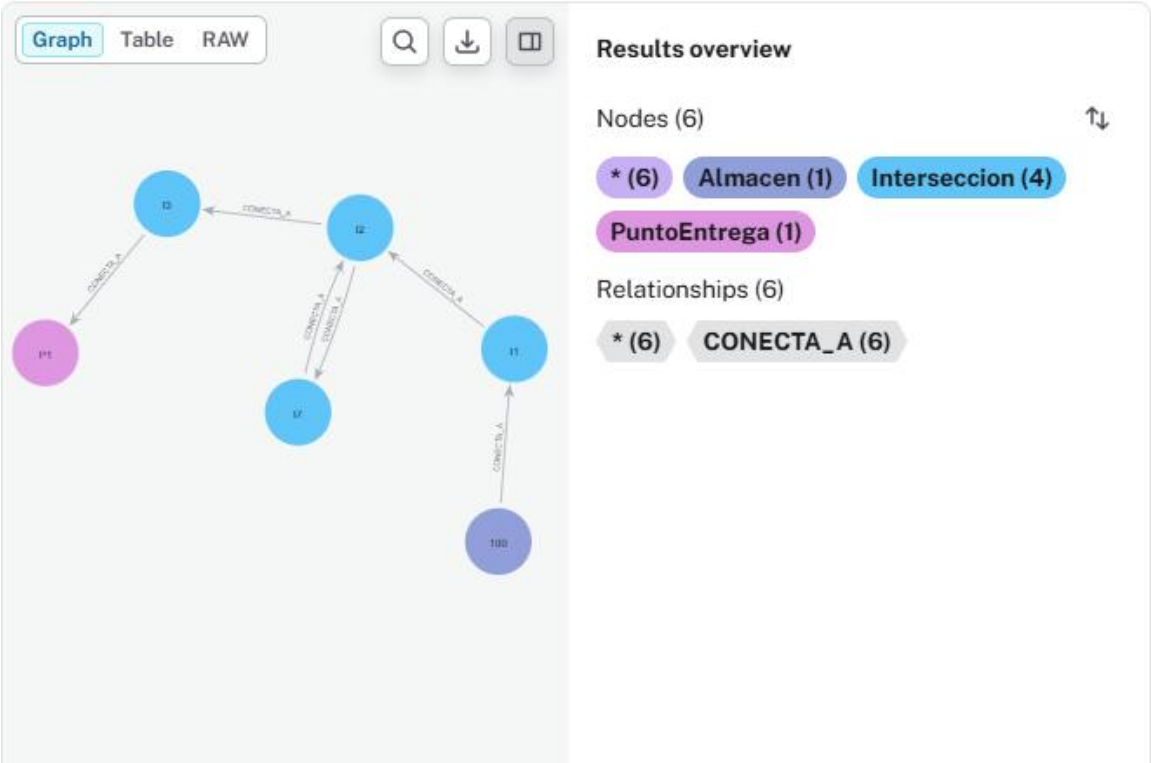
Codigo:

```
// ==========================================
// RUTAS QUE SOPORTEN 15 TONELADAS
// ==========================================

MATCH path = (a:Almacen {id:'A1'})-[:CONECTA_A*]->(p:PuntoEntrega {id:'P1'})
WHERE ALL(r IN relationships(path) WHERE r.peso_maximo >= 15)
RETURN path;
```

Explicación: Usa el predicado ALL() para garantizar que todas las relaciones del camino cumplen la condición peso_maximo >= valor.

Resultado:



path

1 100 - CONECTA_A → I1 - CONECTA_A → I2 - CONECTA_A → I3 - CONECTA_A → P1

2 100 - CONECTA_A → I1 - CONECTA_A → I2 - CONECTA_A → I7 - CONECTA_A → I2 - CONECTA_A → I3 - CONECTA_A → P1

Consulta 5 - Consulta con UNWIND

Objetivo: Examinar cada punto del recorrido de forma individual, descomponer la lista de nodos que forman una ruta calculada por un algoritmo de camino más corto.

Codigo:

// ==========

```
// DESCOMPOSICION DE NODOS (USO DE UNWIND)
// ==========================================

MATCH (source:Almacen {id:'A1'}),
     (target:PuntoEntrega {id:'P1'})

CALL gds.shortestPath.dijkstra.stream('logisticsGraph', {
   sourceNode: id(source),
   targetNode: id(target),
   relationshipWeightProperty: 'distancia'
})
YIELD nodeIds

UNWIND nodeIds AS nodeId
RETURN gds.util.asNode(nodeId).id AS punto_de_ruta;
```

Explicacion: La ruta es devuelta por el algoritmo Dijkstra en forma de una lista de identificadores internos (nodeIds). La cláusula UNWIND convierte esa lista en filas separadas, lo que posibilita manejar cada nodo como un registro autónomo. Después, gds.util.asNode() transforma cada identificador interno en el nodo original para recuperar la propiedad id. Esto posibilita observar el trayecto paso a paso.

Resultado:

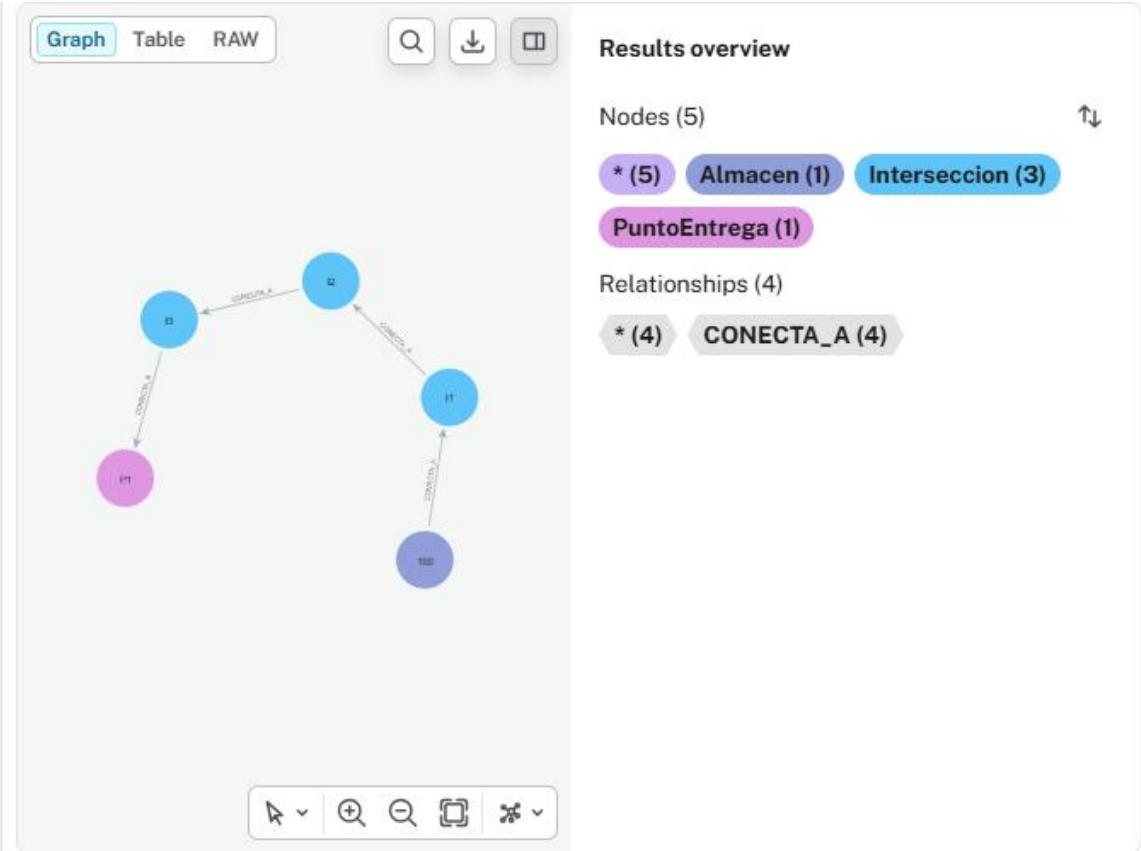
Table RAW

	punto_de_ruta
1	"A1"
2	"I1"
3	"I2"
4	"I3"
5	"P1"

Dashboard de Visualizacion

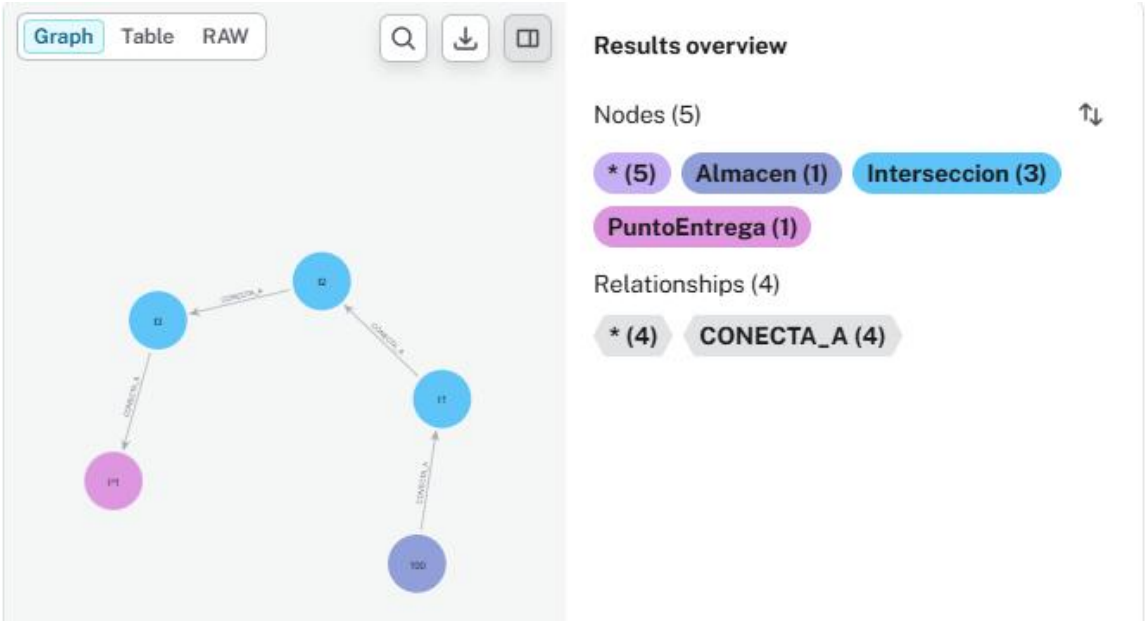
Comparación de resultados entre Dijkstra (basado en distancia) vs A* (enfoque en tiempo/trafico), mediante Neo4j Browser

Peso distancia



algoritmo	costo_total	n1	r	n2
1 "Dijkstra"	13.0	100	CONECTA_A	I1
2 "Dijkstra"	13.0	I1	CONECTA_A	I2
3 "Dijkstra"	13.0	I2	CONECTA_A	I3
4 "Dijkstra"	13.0	I3	CONECTA_A	P1

Peso tiempo_trafico



	algoritmo	costo_total	n1	r	n2
1	"A*"	26.0	100	CONECTA_A	I1
2	"A*"	26.0	I1	CONECTA_A	I2
3	"A*"	26.0	I2	CONECTA_A	I3
4	"A*"	26.0	I3	CONECTA_A	P1

Algoritmo	Costo	Ruta
Dijkstra (peso = distancia)	13.0	100 (almacén) -> conecta_a -> I1 I1 -> conecta_a -> I2 I2 -> conecta_a -> I3 I3 -> conecta_a -> P1
A* (Peso = tiempo_estimado)	26.0	100 (almacén) -> conecta_a -> I1 I1 -> conecta_a -> I2 I2 -> conecta_a -> I3

		I3 -> conecta_a -> P1

Conclusion

El problema principal de este proyecto no es guardar datos aislados, sino examinar las interacciones complejas entre almacenes, cruces y puntos de entrega con el fin de hacer más eficientes las rutas. Esta clase de estructura es muy relacional y necesita varios recorridos entre las entidades interconectadas.

Este modelo requeriría varias tablas intermedias y consultas con múltiples JOIN en una base de datos SQL, lo que incrementa la complejidad y disminuye el rendimiento cuando la red se expande.

Por otro lado, Neo4j posibilita la representación de la red mediante relaciones y nodos, lo que hace más sencillas las consultas de recorrido y la optimización de caminos. También se pueden aplicar algoritmos como A* y Dijkstra sobre el grafo en memoria, con la librería Neo4j Graph Data Science, de forma más eficaz y escalable.

Por eso, para un problema de optimización de rutas en redes urbanas, Neo4j es más apropiado que un modelo relacional convencional.